

Отчет о выполнении задания по параллельному программированию

Вариант 414

Изначальный код:

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#define Max(a,b) ((a)>(b)?(a):(b))

#define N (2*2*2*2*2+2)
double maxeps = 0.1e-7;
int itmax = 100;
int i,j,k;
double eps;
double A [N][N][N], B [N][N][N];

void relax();
void resid();
void init();
void verify();

int main(int an, char **as)
{
    int it;
    init();
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf( "it=%4i  eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    verify();
    return 0;
}

void init()
{
    for(k=0; k<=N-1; k++)
    for(j=0; j<=N-1; j++)
    for(i=0; i<=N-1; i++)
    {
        if(i==0 || i==N-1 || j==0 || j==N-1 || k==0 || k==N-1) A[i][j][k]= 0.;
        else A[i][j][k]= ( 4. + i + j + k );
    }
}

void relax()
{
    for(k=1; k<=N-2; k++)
    for(j=1; j<=N-2; j++)
    for(i=1; i<=N-2; i++)
```

```

{
    B[i][j][k]=(A[i-1][j][k]+A[i+1][j][k]+A[i][j-1][k]+A[i][j+1][k]+A[i][j][k-1]+A[i][j][k+1])/6.;
}
}

void resid()
{
for(k=1; k<=N-2; k++)
for(j=1; j<=N-2; j++)
for(i=1; i<=N-2; i++)
{
    double e;
    e = fabs(A[i][j][k] - B[i][j][k]);
    A[i][j][k] = B[i][j][k];
    eps = Max(eps,e);
}
}

void verify()
{
double s;
s=0.;
for(k=0; k<=N-1; k++)
for(j=0; j<=N-1; j++)
for(i=0; i<=N-1; i++)
{
    s=s+A[i][j][k]*(i+1)*(j+1)*(k+1)/(N*N*N);
}
printf(" S = %f\n",s);
}

```

Изменение кода до применения OpenMP и MPI

I Применено динамическое выделение с линейным представлением для

- возможности работы с различными размерами сетки
- Эффективного использования памяти
- Упрощения передачи матриц в функции:

```

double* allocate_matrix(int N) {
    return (double*)calloc(N * N * N, sizeof(double));
}

```

II Применено чередование матриц для

- Устранение зависимостей по данным между итерациями
- Возможность параллелизации без race conditions:

```

for(int it = 1; it <= itmax; it++) {
    double eps = 0.;
    if (it % 2 == 1) {
        relax(A, B, N, &eps); // A -> B
    } else {
        relax(B, A, N, &eps); // B -> A
    }
}

```

```

    }
    if (eps < maxeps) break;
}

```

III Непосредственное вычисление адреса в линейном массиве для

- Единое представление данных в памяти
- Улучшение производительности за счет локализации данных:

```

A[i][j][k] = (4. + i + j + k);
int idx = i * N * N + j * N + k;
A[idx] = (4. + i + j + k);

```

IV Объединены relax и resid:

```

void relax(double *src, double *dst, int N, double *eps) {
    // одновременное вычисление новых значений и невязки
    double new_val = (соседи) / 6.0;
    dst[idx] = new_val;
    double diff = fabs(src[idx] - new_val);
    // обновление eps
}

```

V Другие изменения:

- 1) Кол-во итераций ограничено 10
- 2) Локализация переменных итераторов
- 3) Все циклы приведены к единому стилю
- 4) Вычисляется время, потраченное на вычисление и верификацию
- 5) Выводиться информация о кол-ве нитей, размере, времени, контрольной сумме

Распараллеливание программы с помощью технологии OpenMP #pragma omp for

Теперь, когда:

- Устранены все зависимости по данным внутри итерации
- Код подготовлен для эффективной параллелизации
- Сохранена математическая корректность алгоритма
- Обеспечена возможность работы с различными размерами задач

Применим **#pragma omp for**:

```

void init(double *A, int N) {
    #pragma omp parallel for collapse(3) schedule(static)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                int idx = i * N * N + j * N + k;
            }
        }
    }
}

```

```

        if(i == 0 || i == N-1 || j == 0 || j == N-1 || k == 0 || k == N-1)
            A[idx] = 0.;
        else
            A[idx] = (4. + i + j + k);
    }
}
}
}

```

```

void relax(double *src, double *dst, int N, double *eps) {
    int num_threads = omp_get_max_threads();
    double *local_eps_array = (double*)calloc(num_threads, sizeof(double));

```

```

#pragma omp parallel
{
    int tid = omp_get_thread_num();
    #pragma omp for collapse(3) schedule(static) nowait
    for(int i = 1; i <= N-2; i++) {
        for(int j = 1; j <= N-2; j++) {
            for(int k = 1; k <= N-2; k++) {
                int idx = i * N * N + j * N + k;

                double new_val = (src[(i-1) * N * N + j * N + k] +
                                   src[(i+1) * N * N + j * N + k] +
                                   src[i * N * N + (j-1) * N + k] +
                                   src[i * N * N + (j+1) * N + k] +
                                   src[i * N * N + j * N + (k-1)] +
                                   src[i * N * N + j * N + (k+1)]) / 6.0;

                dst[idx] = new_val;

                double diff = fabs(src[idx] - new_val);
                if(diff > local_eps_array[tid])
                    local_eps_array[tid] = diff;
            }
        }
    }
}

```

```

// Редукция максимального значения eps
*eps = 0.0;
for(int t = 0; t < num_threads; t++) {
    if(local_eps_array[t] > *eps) {
        *eps = local_eps_array[t];
    }
}
free(local_eps_array);
}

```

```

void verify(double *A, int N, double *s) {
    double local_s = 0.;

    #pragma omp parallel for collapse(3) schedule(static) reduction(+:local_s)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {

```



```

int idx = i * N * N + j * N + k;

double new_val = (src[(i-1) * N * N + j * N + k] +
    src[(i+1) * N * N + j * N + k] +
    src[i * N * N + (j-1) * N + k] +
    src[i * N * N + (j+1) * N + k] +
    src[i * N * N + j * N + (k-1)] +
    src[i * N * N + j * N + (k+1)]) / 6.0;

dst[idx] = new_val;

double diff = fabs(src[idx] - new_val);
if(diff > my_eps) my_eps = diff;
    }
}
local_eps_array[tid] = my_eps;
}
}
#pragma omp taskwait
}
}

*eps = 0.0;
for(int t = 0; t < num_threads; t++) {
    if(local_eps_array[t] > *eps) {
        *eps = local_eps_array[t];
    }
}
free(local_eps_array);
}

```

II Основные моменты:

Создание и управление задачами

- `#pragma omp single` - один поток создает все задачи
- `#pragma omp task` - создание задачи для каждого значения `i`
- `#pragma omp taskwait` - барьер для ожидания завершения всех задач

Распределение данных

- `firstprivate(i)` - приватная копия переменной цикла для каждой задачи
- `shared` массивы - общий доступ к исходным и результирующим данным
- Локальные переменные внутри задачи автоматически приватные

Гранулярность работы

- Одна задача = одна плоскость $i = (N-2) \times (N-2)$ точек
- Всего создается $N-2$ задач
- Крупный размер задач минимизирует накладные расходы

Редукция максимального значения

- Каждая задача вычисляет локальный максимум `my_eps`
- Результаты сохраняются в общий массив `local_eps_array` по индексу потока
- После завершения всех задач выполняется последовательная редукция

Распараллеливание программы с помощью технологии MPI

I Подход к распределению данных

В отличие от OpenMP-версий, где распараллеливание происходило в рамках одной вычислительной системы с общей памятью, MPI-версия реализует распределённые вычисления с передачей данных по сети. Основные особенности:

Декомпозиция по координате Z:

Трёхмерная сетка разделяется между процессами по оси Z (слоям)

Каждый процесс получает непрерывный блок слоёв для обработки

Для корректного вычисления требуется обмен граничными слоями между соседними процессами

Каждый процесс хранит свои слои плюс по одному граничному слою сверху и снизу для обмена с соседями.

II Алгоритм вычислений

Инициализация:

Каждый процесс инициализирует свои слои, включая граничные условия.

Итерационный процесс:

Обмен граничными слоями с соседними процессами:

```
MPI_Sendrecv(&A[1][0][0], N*N, MPI_DOUBLE, up_neigh, 0,
             &A[0][0][0], N*N, MPI_DOUBLE, up_neigh, 1,
             MPI_COMM_WORLD, &status);
```

Вычисление новых значений для внутренних точек:

Используется формула Якоби для всех внутренних слоёв

Учитываются соседние значения, включая полученные граничные слои

III Особенности реализации

Распределение нагрузки:

```
int layers_per_proc = layers / size;
```

```
int extra_layers = layers % size;
```

Распределение происходит с балансировкой нагрузки - первые `extra_layers` процессов получают на один слой больше.

Обмен данными:

Используется синхронный обмен `MPI_Sendrecv` для предотвращения deadlock

Двойная буферизация: массивы A и B чередуются для устранения зависимостей

Tag 0 и 1 используются для различения направлений передачи

Вычисление контрольной суммы:

```
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
           MPI_COMM_WORLD);
```

Частичные суммы вычисляются локально и объединяются на процессе 0.

Запуск и анализ результатов

I Условия запуска:

- Суперкомпьютер: Polus
- Компилятор: xlc_r с поддержкой OpenMP
- Флаги компиляции: -qsmp=omp -qarch=pwr8 -qtune=pwr8
- Уровни оптимизации: -O2, -O3, -O5
- Диапазон потоков: 1, 2, 4, 8, 16, 20, 40, 60, 80, 100, 120, 140, 160
- Размеры сетки: 32×32×32, 64×64×64, 96×96×96, 128×128×128, 192×192×192, 256×256×256

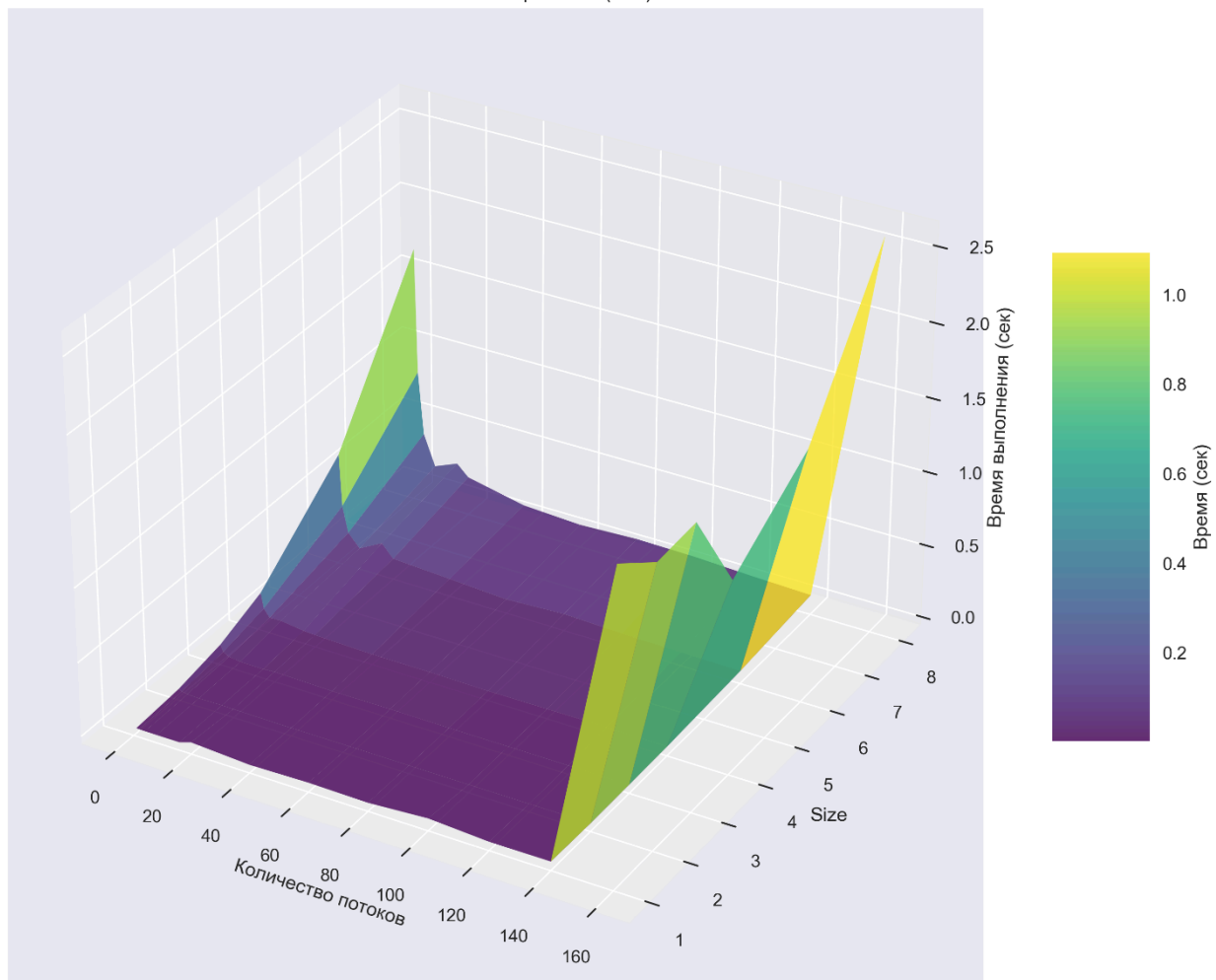
II Измерения

Для каждого сочетания параметров (версия, оптимизация, количество потоков, размер сетки) проводились замеры времени выполнения, включая инициализацию, вычисления и верификацию. Управление размещением потоков настраивалось динамически:

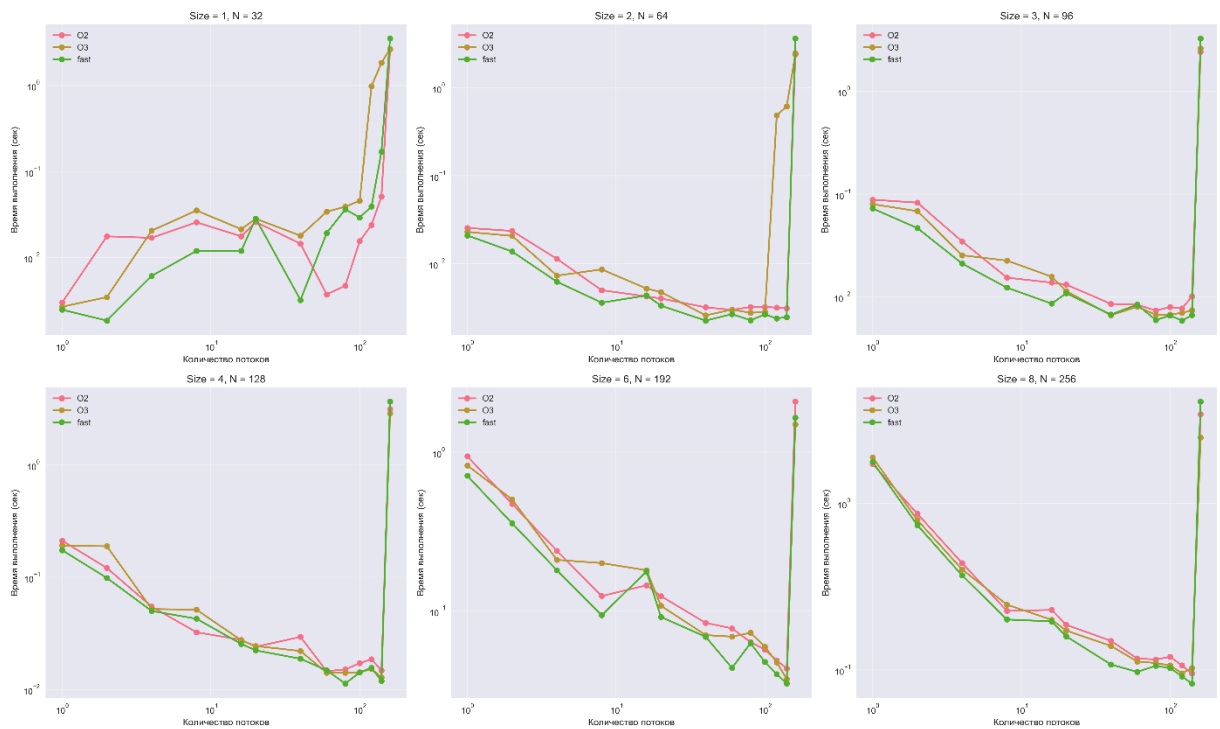
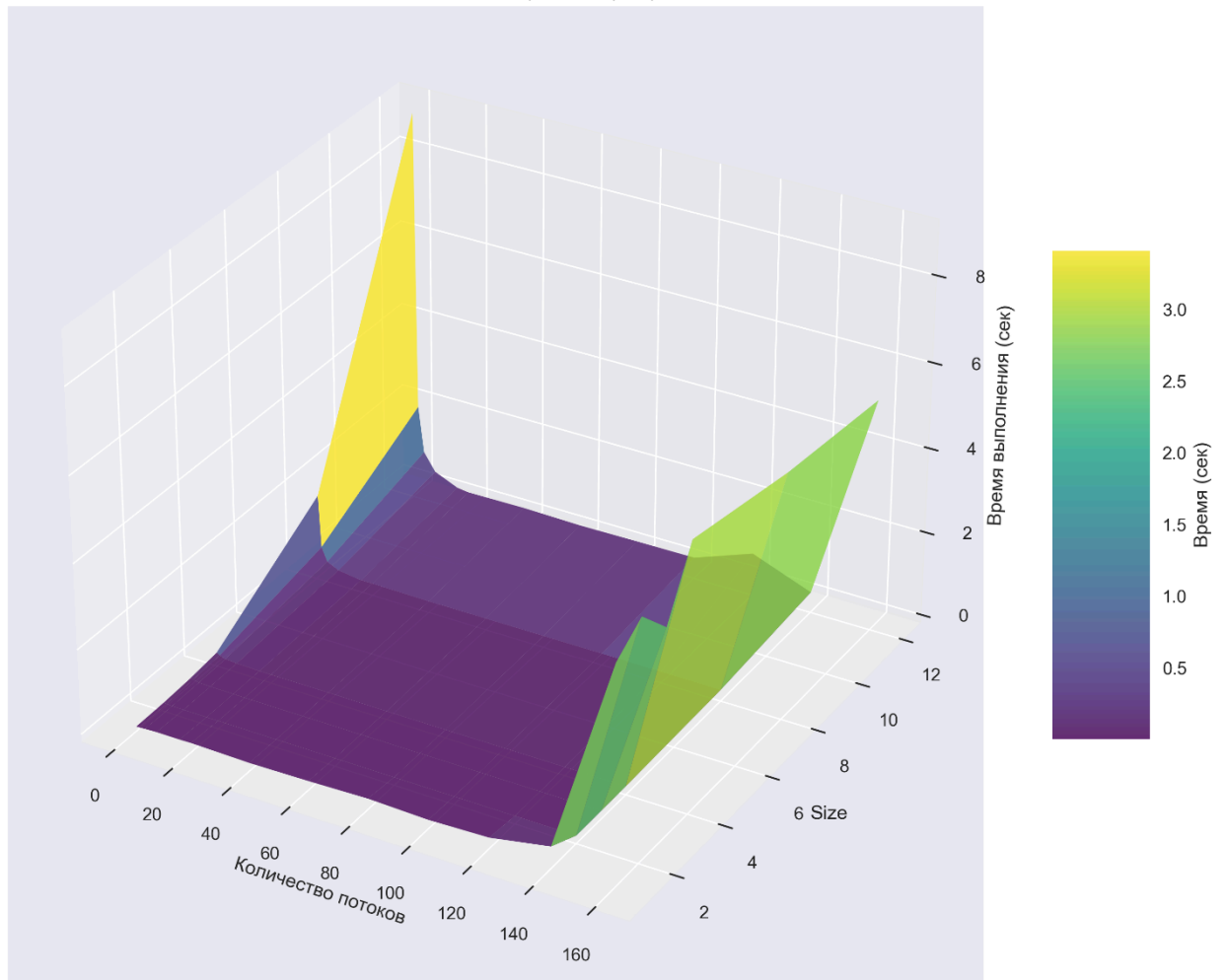
```
if [ $threads -le 20 ]; then
    export OMP_PLACES="cores"
    export OMP_PROC_BIND="close"
elif [ $threads -le 40 ]; then
    export OMP_PLACES="threads"
    export OMP_PROC_BIND="spread,close"
else
    export OMP_PLACES="threads"
    export OMP_PROC_BIND="spread"
fi
```

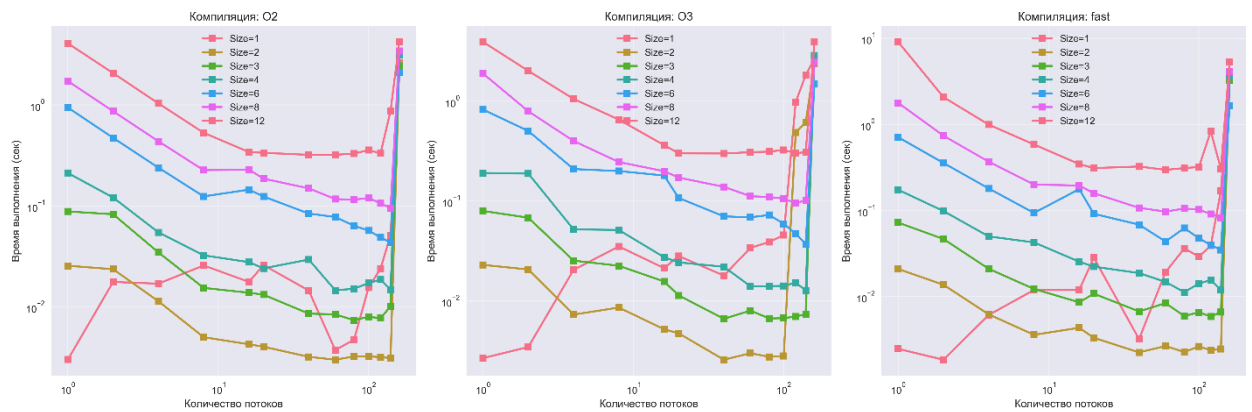

III Результаты на графиках

3D анализ: FOR стратегия (fast)



3D анализ: TASK стратегия (fast)





Стратегия `#pragma omp for`

Показала наилучшие результаты по абсолютной производительности, особенно на больших размерах сетки (256×256×256 и более). Ключевые преимущества:

- Минимальные накладные расходы на синхронизацию
- Эффективное использование кэш-памяти благодаря статическому распределению блоков
- Оптимальное сочетание с оптимизациями компилятора

Стратегия `#pragma omp task`

Обладает лучшей стабильностью работы, но уступает в производительности из-за:

- Дополнительных затрат на создание и управление задачами
- Повышенной сложности механизма планирования

Стратегия MPI

Преимущества:

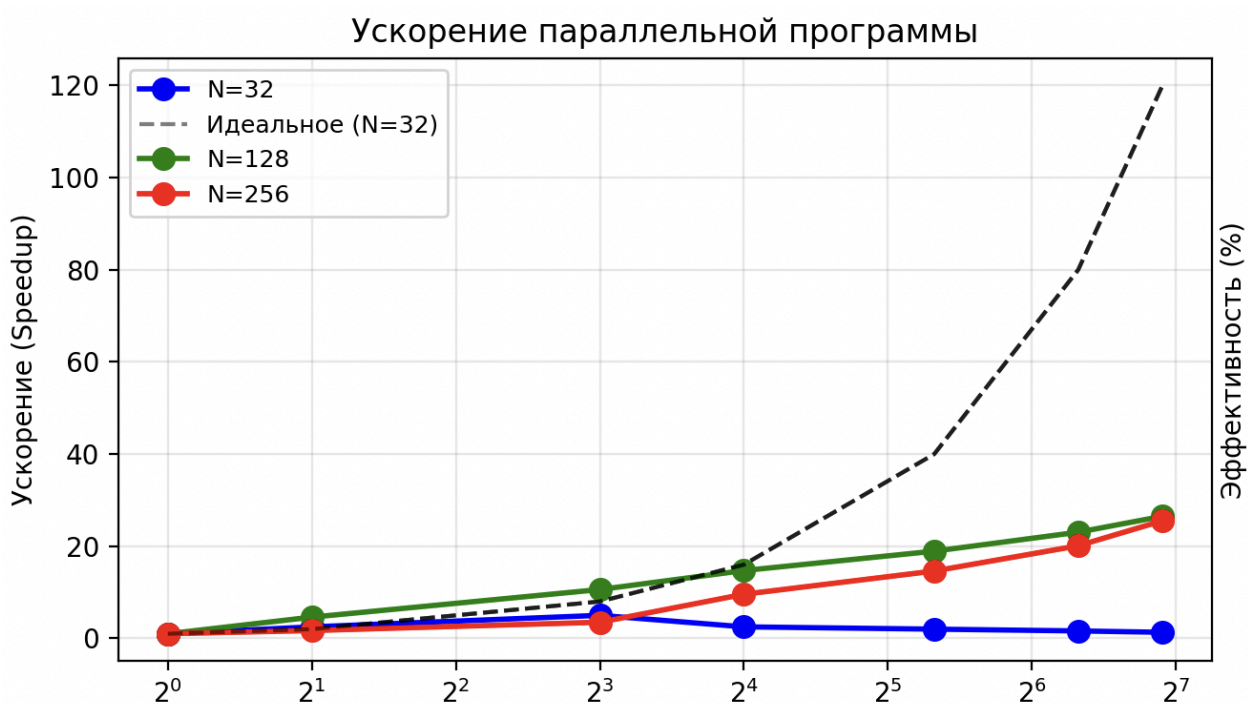
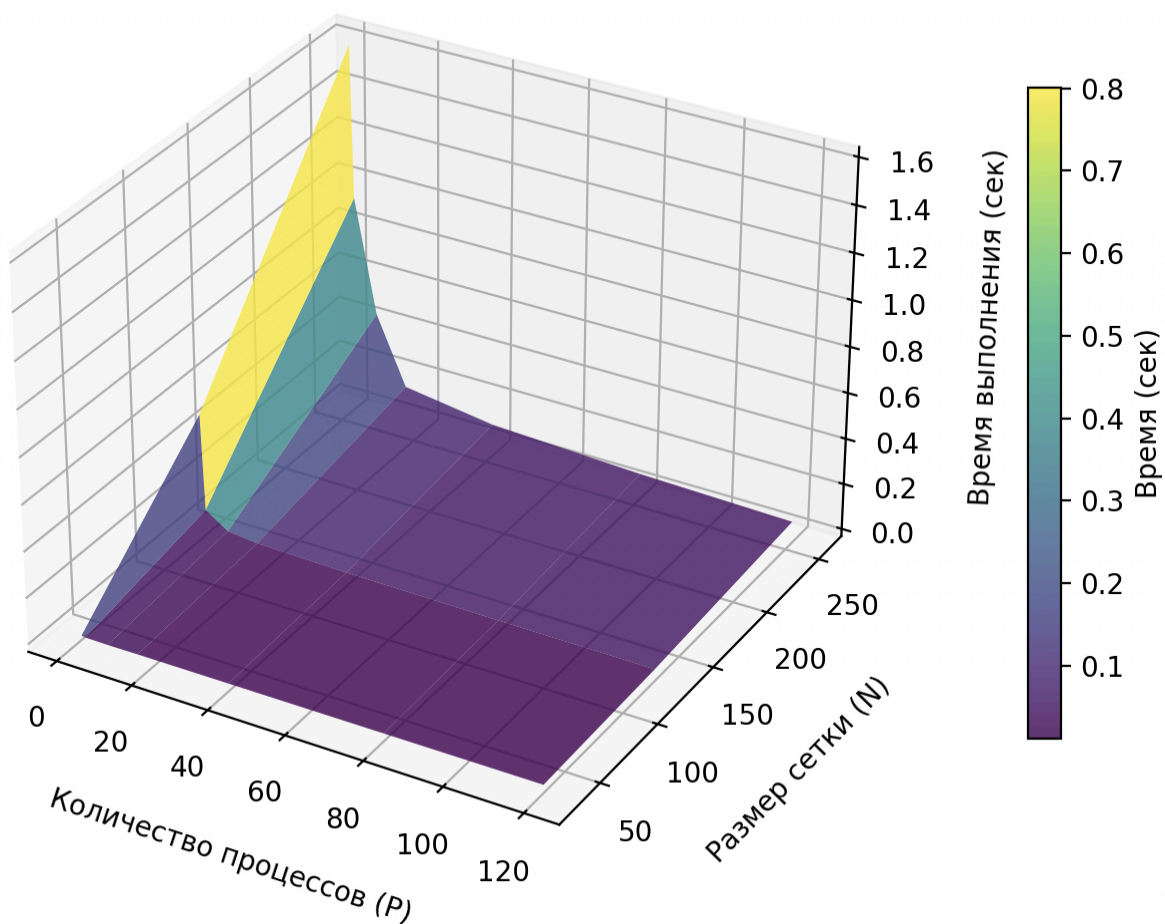
- Возможность масштабирования на сотни и тысячи процессов
- Эффективное использование распределённых вычислительных ресурсов
- Независимость от архитектуры отдельных узлов
- Естественное соответствие геометрической декомпозиции задачи

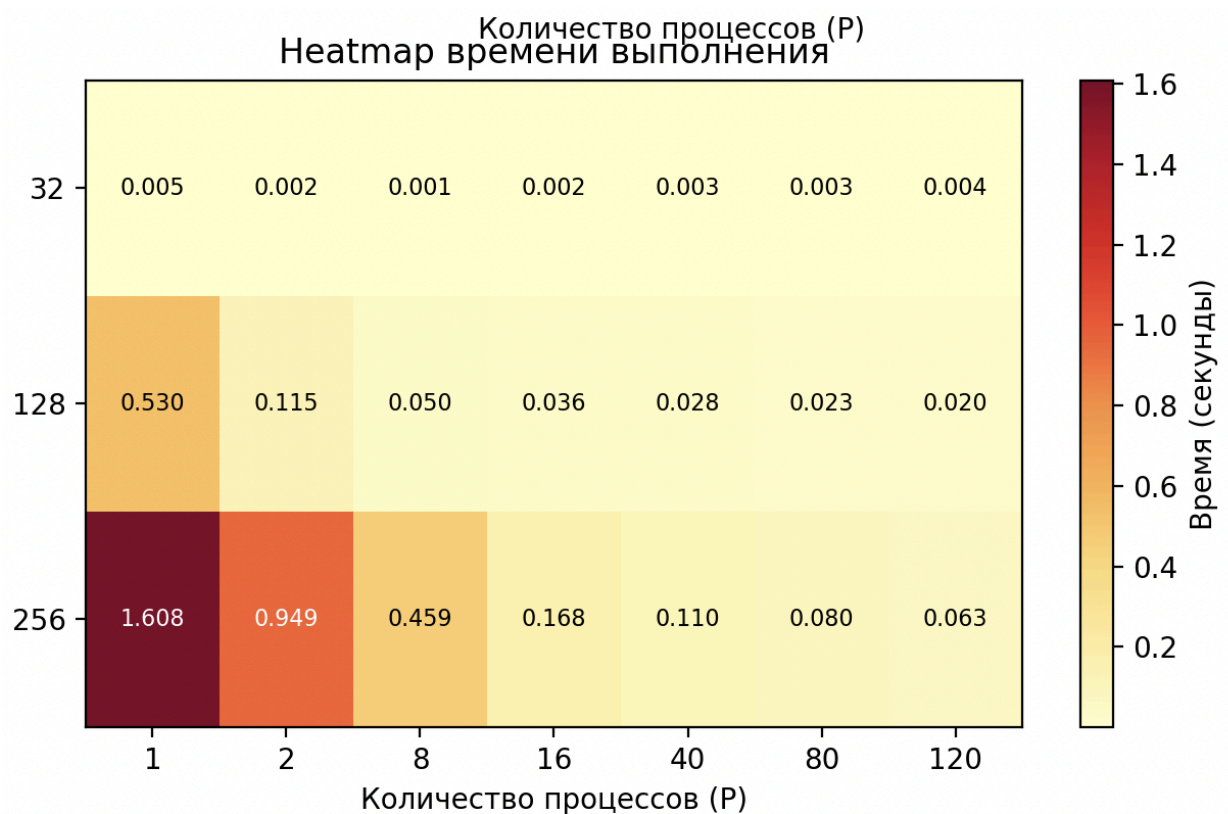
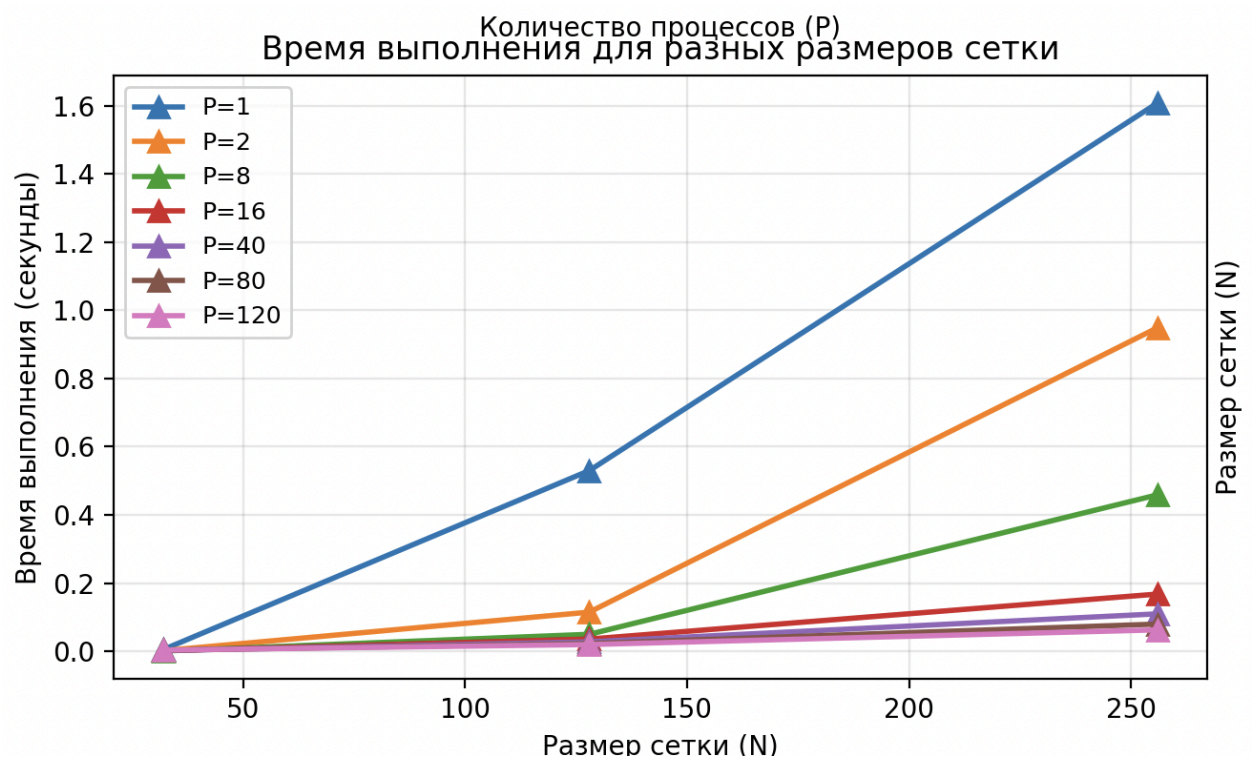
Ограничения:

- Высокие накладные расходы на передачу граничных слоёв
- Сложность динамической балансировки нагрузки
- Необходимость ручного управления распределением данных
- Ограниченная эффективность при малом числе слоёв на процесс

Графики

(Диапазоны параметров уменьшены по причине загруженности Полюса)





Общий вывод:

Архитектурные ограничения системы проявляются при использовании более 60 потоков, где накладные расходы на синхронизацию и конкуренция за ресурсы памяти превосходят выгоду от параллелизма.

Размер задачи напрямую влияет на эффективность параллелизации - большие сетки (256×256×256 и более) лучше масштабируются и демонстрируют более предсказуемое поведение.

Пороговый эффект при 120-160 потоках указывает на исчерпание физических ресурсов системы и необходимость использования более сложных стратегий распределения работы для экстремального параллелизма.

Рекомендуется использование стратегии for с количеством потоков не более 40 для достижения оптимального баланса между производительностью и стабильностью выполнения.

MPI-версия демонстрирует принципиально иной подход к параллелизации - через распределение данных и вычислений между независимыми процессами.

MPI эффективен для очень больших задач, где объём вычислений значительно превышает накладные расходы на передачу данных.

Для данной вычислительной задачи оптимальной является стратегия for с количеством потоков 16-40, что соответствует физическим возможностям аппаратной платформы.

Все программы:

I FOR

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>

#define Max(a,b) ((a)>(b)?(a):(b))

double maxeps = 0.1e-7;
int itmax = 10;

void relax(double *src, double *dst, int N, double *eps);
void init(double *A, int N);
void verify(double *A, int N, double *s);
double* allocate_matrix(int N);
void free_matrix(double *A);

int main(int an, char **as)
{
    int num_threads = 1;
    if (an > 1) {
        num_threads = atoi(as[1]);
    }
    omp_set_num_threads(num_threads);

    int base_N = 32;
```

```

int sizes[] = {1, 2, 3, 4, 6, 8};
int num_sizes = 6;

printf("Threads,Size,N,Time,Sum,Version\n");

for (int size_idx = 0; size_idx < num_sizes; size_idx++) {
    int N = base_N * sizes[size_idx];
    double *A = allocate_matrix(N);
    double *B = allocate_matrix(N);

    double start_time = omp_get_wtime();

    init(A, N);

    for(int it = 1; it <= itmax; it++) {
        double eps = 0.;
        if (it % 2 == 1) {
            relax(A, B, N, &eps);
        } else {
            relax(B, A, N, &eps);
        }
        if (eps < maxeps) break;
    }

    double s;
    verify(A, N, &s);
    double end_time = omp_get_wtime();

    printf("%d,%d,%d,%.6f,%.6f,for\n",
        num_threads, sizes[size_idx], N, end_time - start_time, s);

    free_matrix(A);
    free_matrix(B);
}

return 0;
}

double* allocate_matrix(int N) {
    return (double*)calloc(N * N * N, sizeof(double));
}

void free_matrix(double *A) {
    free(A);
}

void init(double *A, int N) {
    #pragma omp parallel for collapse(3) schedule(static)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                int idx = i * N * N + j * N + k;
                if(i == 0 || i == N-1 || j == 0 || j == N-1 || k == 0 || k == N-1)
                    A[idx] = 0.;
                else
                    A[idx] = (4. + i + j + k);
            }
        }
    }
}

```

```

    }
  }
}

void relax(double *src, double *dst, int N, double *eps) {
  int num_threads = omp_get_max_threads();
  double *local_eps_array = (double*)calloc(num_threads, sizeof(double));

  #pragma omp parallel
  {
    int tid = omp_get_thread_num();
    #pragma omp for collapse(3) schedule(static) nowait
    for(int i = 1; i <= N-2; i++) {
      for(int j = 1; j <= N-2; j++) {
        for(int k = 1; k <= N-2; k++) {
          int idx = i * N * N + j * N + k;

          double new_val = (src[(i-1) * N * N + j * N + k] + src[(i+1) * N * N + j * N + k] +
                           src[i * N * N + (j-1) * N + k] + src[i * N * N + (j+1) * N + k] +
                           src[i * N * N + j * N + (k-1)] + src[i * N * N + j * N + (k+1)]) / 6.;

          dst[idx] = new_val;

          double diff = fabs(src[idx] - new_val);
          if(diff > local_eps_array[tid]) local_eps_array[tid] = diff;
        }
      }
    }
  }

  *eps = 0.0;
  for(int t = 0; t < num_threads; t++) {
    if(local_eps_array[t] > *eps) {
      *eps = local_eps_array[t];
    }
  }
  free(local_eps_array);
}

void verify(double *A, int N, double *s) {
  double local_s = 0.;

  #pragma omp parallel for collapse(3) schedule(static) reduction(+:local_s)
  for(int i = 0; i < N; i++) {
    for(int j = 0; j < N; j++) {
      for(int k = 0; k < N; k++) {
        int idx = i * N * N + j * N + k;
        local_s += A[idx] * (i+1) * (j+1) * (k+1) / (N*N*N);
      }
    }
  }

  *s = local_s;
}

```


II TASK

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>

#define Max(a,b) ((a)>(b)?(a):(b))

double maxeps = 0.1e-7;
int itmax = 10;

void relax(double *src, double *dst, int N, double *eps);
void init(double *A, int N);
void verify(double *A, int N, double *s);
double* allocate_matrix(int N);
void free_matrix(double *A);

int main(int an, char **as)
{
    int num_threads = 1;
    if (an > 1) {
        num_threads = atoi(as[1]);
    }
    omp_set_num_threads(num_threads);

    int base_N = 32;
    int sizes[] = {1, 2, 3, 4, 8, 12};
    int num_sizes = 6;

    printf("Threads,Size,N,Time,Sum,Version\n");

    for (int size_idx = 0; size_idx < num_sizes; size_idx++) {
        int N = base_N * sizes[size_idx];
        double *A = allocate_matrix(N);
        double *B = allocate_matrix(N);

        double start_time = omp_get_wtime();

        init(A, N);

        for(int it = 1; it <= itmax; it++) {
            double eps = 0.;
            if (it % 2 == 1) {
                relax(A, B, N, &eps);
            } else {
                relax(B, A, N, &eps);
            }
            if (eps < maxeps) break;
        }

        double s;
        verify(A, N, &s);
        double end_time = omp_get_wtime();

        printf("%d,%d,%d,%.6f,%.6f,task\n",
```

```

        num_threads, sizes[size_idx], N, end_time - start_time, s);

    free_matrix(A);
    free_matrix(B);
}

return 0;
}

double* allocate_matrix(int N) {
    return (double*)calloc(N * N * N, sizeof(double));
}

void free_matrix(double *A) {
    free(A);
}

void init(double *A, int N) {
    #pragma omp parallel for collapse(3) schedule(static)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                int idx = i * N * N + j * N + k;
                if(i == 0 || i == N-1 || j == 0 || j == N-1 || k == 0 || k == N-1)
                    A[idx] = 0.;
                else
                    A[idx] = (4. + i + j + k);
            }
        }
    }
}

void relax(double *src, double *dst, int N, double *eps) {
    int num_threads = omp_get_max_threads();
    double *local_eps_array = (double*)calloc(num_threads, sizeof(double));

    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        #pragma omp single
        {
            for(int i = 1; i <= N-2; i++) {
                #pragma omp task firstprivate(i) shared(local_eps_array, tid, src, dst)
                {
                    double my_eps = 0.0;
                    for(int j = 1; j <= N-2; j++) {
                        for(int k = 1; k <= N-2; k++) {
                            int idx = i * N * N + j * N + k;

                            double new_val = (src[(i-1) * N * N + j * N + k] + src[(i+1) * N * N + j * N + k] +
                                src[i * N * N + (j-1) * N + k] + src[i * N * N + (j+1) * N + k] +
                                src[i * N * N + j * N + (k-1)] + src[i * N * N + j * N + (k+1)]) / 6.0;

                            dst[idx] = new_val;

                            double diff = fabs(src[idx] - new_val);

```

```

        if(diff > my_eps) my_eps = diff;
    }
}
local_eps_array[tid] = my_eps;
}
}
#pragma omp taskwait
}
}

*eps = 0.0;
for(int t = 0; t < num_threads; t++) {
    if(local_eps_array[t] > *eps) {
        *eps = local_eps_array[t];
    }
}
free(local_eps_array);
}

void verify(double *A, int N, double *s) {
    double local_s = 0.;

    #pragma omp parallel for collapse(3) schedule(static) reduction(+:local_s)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                int idx = i * N * N + j * N + k;
                local_s += A[idx] * (i+1) * (j+1) * (k+1) / (N*N*N);
            }
        }
    }

    *s = local_s;
}

```

III MPI

```

#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <mpi.h>
#include <time.h>

int N = 128;
int itmax = 10;
double maxeps = 1e-7;

int main(int argc, char **argv) {
    int rank, size;
    double start_time, end_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

```

```

for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "-n") == 0 && i+1 < argc) {
        N = atoi(argv[++i]);
    } else if (strcmp(argv[i], "-it") == 0 && i+1 < argc) {
        itmax = atoi(argv[++i]);
    }
}

if (rank == 0) {
    printf("=== MPI JACOBI SCALING ===\n");
    printf("Grid: %d^3, Processes: %d, Iterations: %d\n", N, size, itmax);
    start_time = MPI_Wtime();
}

int layers = N - 2;
int layers_per_proc = layers / size;
int extra_layers = layers % size;

int start_layer = 1;
for (int i = 0; i < rank; i++) {
    start_layer += layers_per_proc + (i < extra_layers ? 1 : 0);
}

int end_layer = start_layer + layers_per_proc + (rank < extra_layers ? 1 : 0);
int my_layers = end_layer - start_layer;

int total_layers = my_layers + 2;
double (*A)[N][N] = malloc(total_layers * sizeof(*A));
double (*B)[N][N] = malloc(total_layers * sizeof(*A));

if (A == NULL || B == NULL) {
    fprintf(stderr, "Process %d: Memory allocation failed!\n", rank);
    MPI_Abort(MPI_COMM_WORLD, 1);
}

for (int k = 0; k < total_layers; k++) {
    int global_k = start_layer - 1 + k;
    for (int j = 0; j < N; j++) {
        for (int i = 0; i < N; i++) {
            if (i == 0 || i == N-1 || j == 0 || j == N-1 ||
                global_k == 0 || global_k == N-1) {
                A[k][j][i] = 0.0;
            } else {
                A[k][j][i] = 4.0 + i + j + global_k;
            }
        }
    }
}

double eps;
for (int it = 1; it <= itmax; it++) {
    double local_eps = 0.0;

```

```

int up_neigh = (rank > 0) ? rank - 1 : MPI_PROC_NULL;
int down_neigh = (rank < size - 1) ? rank + 1 : MPI_PROC_NULL;

MPI_Status status;

if (up_neigh != MPI_PROC_NULL) {
    MPI_Sendrecv(&A[1][0][0], N*N, MPI_DOUBLE, up_neigh, 0,
                 &A[0][0][0], N*N, MPI_DOUBLE, up_neigh, 1,
                 MPI_COMM_WORLD, &status);
}

if (down_neigh != MPI_PROC_NULL) {
    MPI_Sendrecv(&A[total_layers-2][0][0], N*N, MPI_DOUBLE, down_neigh, 1,
                 &A[total_layers-1][0][0], N*N, MPI_DOUBLE, down_neigh, 0,
                 MPI_COMM_WORLD, &status);
}

for (int k = 1; k <= total_layers - 2; k++) {
    for (int j = 1; j < N-1; j++) {
        for (int i = 1; i < N-1; i++) {
            B[k][j][i] = (A[k-1][j][i] + A[k+1][j][i] +
                          A[k][j-1][i] + A[k][j+1][i] +
                          A[k][j][i-1] + A[k][j][i+1]) / 6.0;
        }
    }
}

for (int k = 1; k <= total_layers - 2; k++) {
    for (int j = 1; j < N-1; j++) {
        for (int i = 1; i < N-1; i++) {
            double e = fabs(A[k][j][i] - B[k][j][i]);
            A[k][j][i] = B[k][j][i];
            if (e > local_eps) local_eps = e;
        }
    }
}

MPI_Allreduce(&local_eps, &eps, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);

if (eps < maxeps) break;
}

double local_sum = 0.0;
for (int k = 1; k <= total_layers - 2; k++) {
    int global_k = start_layer + k - 2;
    for (int j = 0; j < N; j++) {
        for (int i = 0; i < N; i++) {
            local_sum += A[k][j][i] * (i+1) * (j+1) * (global_k+1) / (N*N*N);
        }
    }
}
}

```

```
double global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    end_time = MPI_Wtime();
    double elapsed = end_time - start_time;

    printf("RESULTS: S=%.6f, Time=%.3fs, N=%d, P=%d\n",
          global_sum, elapsed, N, size);
}

free(A);
free(B);
MPI_Finalize();
return 0;
}
```